UNIVERSITÄT PASSAU
Fakultät für Informatik und Mathematik

# Dependable Distributed Systems – 5880V/UE

04. Broadcast semantics and algorithms – 2026-11-02

Prof. Dr. Hans P. Reiser | WS 2026/27

UNIVERSITÄT PASSAU



# Overview


1. Basics: Safety and Liveness

2. Broadcast
   - Best-effort broadcast
   - Reliable broadcast (rbcast)
   - FIFO broadcast (fbcast)
   - Causal broadcast (cbcast)
   - Atomic broadcast (abcast)

# Safety and liveness [according to Lamport]

- Safety *(Sicherheit)*
  - **Something bad does not happen**
  - Safety property defines that some events must *never* happen
  - In other words: Safety property is a predicate $P$ that is always true

- Liveness *(Lebendigkeit)*
  - **Something good will happen**
  - Liveness property defines good events that at some time (“eventually”) must happen
  - In other words: Liveness property is a predicate $P$ that eventually becomes true

# Safety and liveness

## Beware of false friends

"eventually" is (close to) the opposite of the german word "eventuell"

- *"X eventually happens"* means that:
- *"X **will happen** within an unspecified but finite amount of time"*

# Safety and liveness

Example: Factorization algorithm $Fac(n) \rightarrow \{p_i\}$:

- If $Fac(n)$ is started with an argument $n$ that is the product of two prime numbers, $Fac$ will output $p_1 > 1, p_2 > 1$ with $p_1 \cdot p_2 = n$.

# Safety and liveness

Example: Factorization algorithm $Fac(n) \rightarrow \{p_i\}$:

- If $Fac(n)$ is started with an argument $n$ that is the product of two prime numbers, $Fac$ will output $p_1 > 1, p_2 > 1$ with $p_1 \cdot p_2 = n$.
- Safety property or liveness property?

# Safety and liveness

Verification usually is easier if you use multiple minimalistic properties!

Example:

- *Liveness:* The algorithm $Fac(n)$, started with an arbitrary integer $n$ as argument, eventually terminates.
- *Safety:* If $Fac(n)$ terminates and produces output $\{p_1, \ldots, p_k\}$, then $\forall i \in \{1..k\} : p_i$ is a prime number $> 1$ and $\prod_{i=1}^{k} p_i = n$

# Overview

1 Basics: Safety and Liveness

2 Broadcast
- Best-effort broadcast
- Reliable broadcast (rbcast)
- FIFO broadcast (fbcast)
- Causal broadcast (cbcast)
- Atomic broadcast (abcast)

# Broadcasts

## Fault-Tolerant Broadcasts

- Very fundamental abstraction for distributed systems
- A large variety of broadcasts with different **semantics**
- Algorithms highly depend on **system model**

# Broadcasts

## Fault-Tolerant Broadcasts

- Very fundamental abstraction for distributed systems
- A large variety of broadcasts with different **semantics**
- Algorithms highly depend on **system model**

This chapter is heavily based

- on the publication *"A modular approach to fault-tolerant broadcasts and related problems"* by Vassos Hadzilacos and Sam Toueg [Hadzilacos and Toueg, 1994]
  - All following definitions (green boxes) are direct citations from the publication; you should read the article at least up to page 16.
- and on RSDP [Cachin et al., 2011]

# Broadcasts: layered architecture

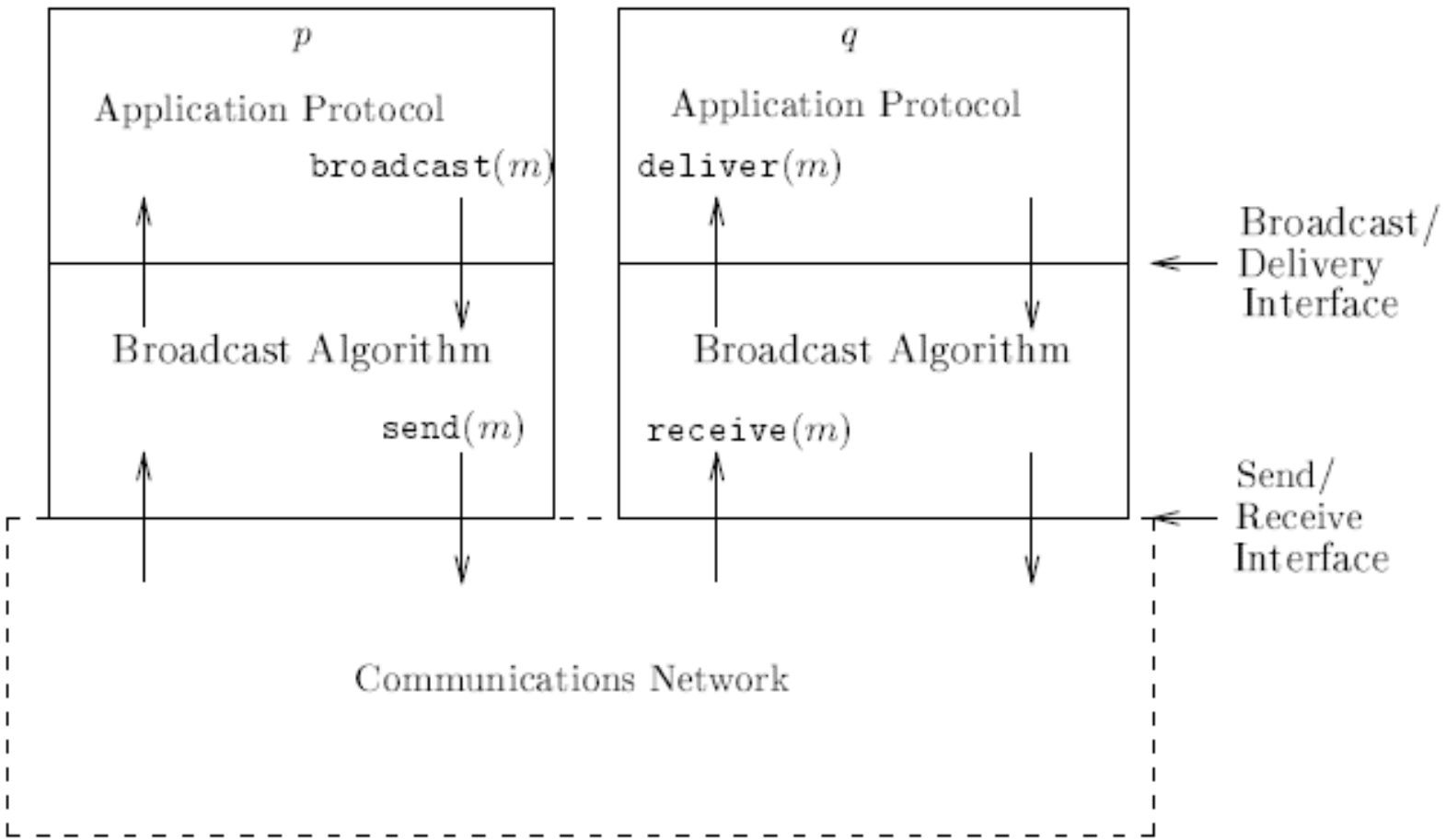



Figure 3: Application/Broadcast Layering

# Broadcast semantics

- Best-effort broadcast
- **rbcast**: Reliable broadcast
- **fbcast**: FIFO broadcast
- **cbcast**: Causal broadcast
- **abcast**: Atomic (total-order) broadcast
- **fabcast**: FIFO atomic broadcast
- **cabcast**: Causal atomic broadcast
- Timed broadcast
- Uniform broadcast

# Best-effort broadcast

Implementation

- Send message to all nodes using perfect point-to-point links
- (Or use existing low-level broadcast mechanisms, such as Ethernet broadcast)

Characteristics

- Simple, low latency
- No message order guarantees
- Reliability?

# Best-effort broadcast

Implementation

- Send message to all nodes using perfect point-to-point links
- (Or use existing low-level broadcast mechanisms, such as Ethernet broadcast)

Characteristics

- Simple, low latency
- No message order guarantees
- Reliable delivery if sender does not crash
  - No guarantees if sender crashes! ($\rightarrow$ see next slide)

# Best-effort broadcast

Best-effort broadcast – $P$ denotes the set of all processes:

```
Uses: PerfectPointToPointLink (ppp)

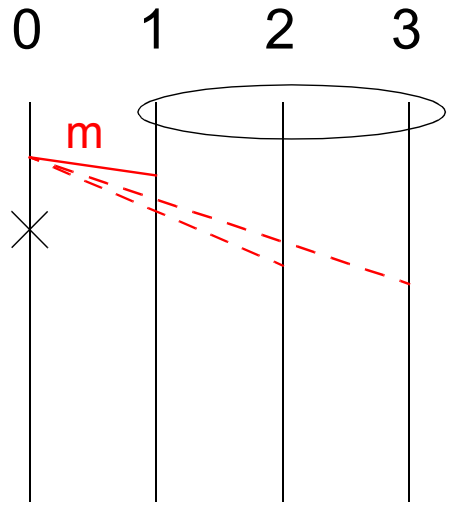
Upon event < BEBroadcast, m > do:
   forall p in P do:
      trigger < ppp.Send, p, m >;

Upon event < ppp.Receive, m > do:
   trigger < BEDeliver, m >;
```

# Best-effort broadcast

Simple algorithm (best-effort broadcast):

- Works fine as long as sender does not crash
- If sender crashes, some but not all processes might receive message

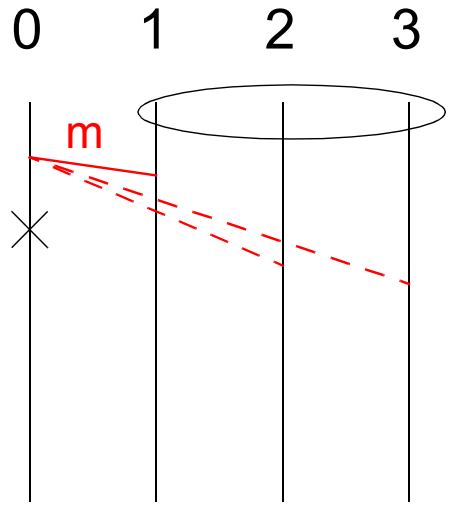


# Best-effort broadcast

Simple algorithm (best-effort broadcast):

- Works fine as long as sender does not crash
- If sender crashes, some but not all processes might receive message

- In some situations, this can be a problem . . .



# Overview


1 Basics: Safety and Liveness

2 Broadcast
- Best-effort broadcast
- Reliable broadcast (rbcast)
- FIFO broadcast (fbcast)
- Causal broadcast (cbcast)
- Atomic broadcast (abcast)

# Reliable broadcast

Basic idea (very informal!):

- Make sure that messages are not lost: all (nonfaulty) processes shall deliver all messages
- No guarantees on message order

# Reliable broadcast

Basic idea (very informal!):

- Make sure that messages are not lost: all (nonfaulty) processes shall deliver all messages
- No guarantees on message order

- Before defining a “correct” algorithm, let’s be a bit more formal. . .

# Reliable broadcast (rbcast)

- Sender $p$ executes operation $broadcast(m)$ with message $m$
  - $m$ contains sender ID and unique sequence number of the sender
  - i.e. $m := (sender, seq\#, data)$
- Message $m$ is delivered at $q$: Operation $deliver(m)$ is executed

## Properties of rbcast

- **Validity:** correct $p$ : $broadcast(m) \Rightarrow p : deliver(m)$ within finite time
- **Agreement:** $\exists$ correct $p$ : $deliver(m) \Rightarrow \forall$ correct $q$ : $deliver(m)$ within finite time
- **Integrity:** Each message $m$ is delivered at most once by each correct process, and only if $sender(m)$ did broadcast the message

# Reliable broadcast (rbcast)

- Think about an algorithm that solves the problem

# Reliable broadcast (rbcast)

- Think about an algorithm that solves the problem

Echo algorithm (from paper):

*Algorithm for process $p$:*
*To execute* `broadcast(R,`$m$`)`:
    `send(`$m$`) to` $p$

`deliver(R,`$m$`)` *occurs as follows:*
    **upon** `receive(`$m$`)` **do**
        **if** $p$ has not previously executed `deliver(R,`$m$`)`
        **then**
            `send(`$m$`) to` all neighbors
            `deliver(R,`$m$`)`

Figure 11: Reliable Broadcast for Point-to-Point Networks

# Reliable broadcast (rbcast)

In our notation:

```
Uses: PerfectPointToPointLink (ppp)

upon Init:
   delivered := empty set;

upon event < RBroadcast, m > do:
   trigger < ppp.Send, self, m >;

upon event < ppp.Receive, m > do:
   if(not delivered.contains(m)) then
      forall p in P do:
         trigger < ppp.Send, p, m >;
      delivered.add(m);
      trigger < RDeliver, m >;
```

# Reliable broadcast (rbcast)

Correctness verification (informal):

- Validity: RBroadcast $\rightarrow$ Send $\rightarrow$ Receive $\rightarrow$ RDeliver
  within finite time (assuming that ppp channel delivers message within finite time)
- Agreement: If some correct process triggers RDeliver, it has triggered ppp.Send for all destinations before. Thus, assuming the properties of perfect links, each correct process $p_i$ will trigger the corresponding ppp.Receive
  - If $m$ is not in delivered at $p_i$, $p_i$ will RDeliver the message
  - If $m$ is in delivered, it was added to delivered before, which implies RDeliver of $m$
- Integrity:
  - the `delivered` set prevents duplication
  - ppp.Receive happens only after ppp.Send (property of ppp link), which in turn happens only after RBroadcast

# Uniform reliable broadcast

- The reliable broadcast so far places no restrictions on messages **delivered by faulty processes**
  - For instance the *agreement* property allows faulty processes to deliver different messages
  - Problematic especially if processes interact with external entities

- Properties can be strengthened ($\rightarrow$ **uniformity**):

## Modified properties of uniform rbcast

- **Uniform agreement (einheitliche Übereinstimmung):**
  $\exists p : deliver(m) \Rightarrow \forall$ correct $q : deliver(m)$ within finite time
- **Uniform integrity (einheitliche Integrität):**
  Each message $m$ is delivered only once, and only if $m$ was sent by $sender(m)$ before.

# Uniform reliable broadcast

Previous echo algorithm:

- Satisfies uniform integrity
- Does **not** satisfy uniform agreement!

# Uniform reliable broadcast

Previous echo algorithm:

- Satisfies uniform integrity
- Does **not** satisfy uniform agreement!

*Note (without proof): It is impossible to implement uniform reliable broadcast in an asynchronous* system model with an arbitrary number of faulty processes!

- Solutions exist for synchronous model
- Solutions exist for asynchronous model if a majority of processes are correct

# Overview


1 Basics: Safety and Liveness

2 Broadcast
- Best-effort broadcast
- Reliable broadcast (rbcast)
- FIFO broadcast (fbcast)
- Causal broadcast (cbcast)
- Atomic broadcast (abcast)

# FIFO broadcast (fbcast)

Difference: order of messages

- Reliable broadcasts: All correct processes deliver all messages, but order may be completely different (i.e., no order guarantees at all)
- FIFO: If $P_i$ broadcasts $m_1$ before $m_2$, then no correct process will deliver $m_2$ unless it has previously delivered $m_1$
- Causal: If $m_1$ causally precedes $m_2$, then no correct process will deliver $m_2$ unless it has previously delivered $m_1$
- Total: If two correct processes, $A$ and $B$, deliver two messages $m_1$ and $m_2$, then $B$ delivers $m_2$ after $m_1$ if and only if $A$ delivers $m_2$ after $m_1$

# FIFO broadcast (fbcast)

Difference: order of messages

- Reliable broadcasts: All correct processes deliver all messages, but order may be completely different (i.e., no order guarantees at all)
- FIFO: If $P_i$ broadcasts $m_1$ before $m_2$, then no correct process will deliver $m_2$ unless it has previously delivered $m_1$
- Causal: If $m_1$ causally precedes $m_2$, then no correct process will deliver $m_2$ unless it has previously delivered $m_1$
- Total: If two correct processes, $A$ and $B$, deliver two messages $m_1$ and $m_2$, then $B$ delivers $m_2$ after $m_1$ if and only if $A$ delivers $m_2$ after $m_1$
- (and combinations FIFO+Total, Causal+Total)

# FIFO broadcast (fbcast)

- Add another property to the definition of reliable broadcast:

## Additional property of fbcast

- **FIFO order:** If a process broadcasts a message $m$ before it broadcasts a message $m'$, then no correct process delivers $m'$ unless it has previously delivered $m$.

- How to implement it?
- We present a *transformation* of reliable broadcast (any algorithm) into FIFO broadcast

# FIFO broadcast (fbcast)

```
Uses: Reliable broadcast (rbcast)

upon event Init:
   msgSet := ∅
   next[s] := 1 for each process s

upon event < FBroadcast, m >:
   trigger < rbcast.RBroadcast, m >;

upon event < rbcast.RDeliver, m >:
   s := sender(m);
   if next[s] = seq#(m) then
      trigger < Fdeliver, m >;
      next[s] := next[s] + 1;
      while(∃ m' in msgSet: sender(m')=s and next[s]=seq#(m')):
          trigger < Fdeliver, m' >;
          next[s] := next[s] + 1;
   else
      msgSet := msgSet ∪ m;
```

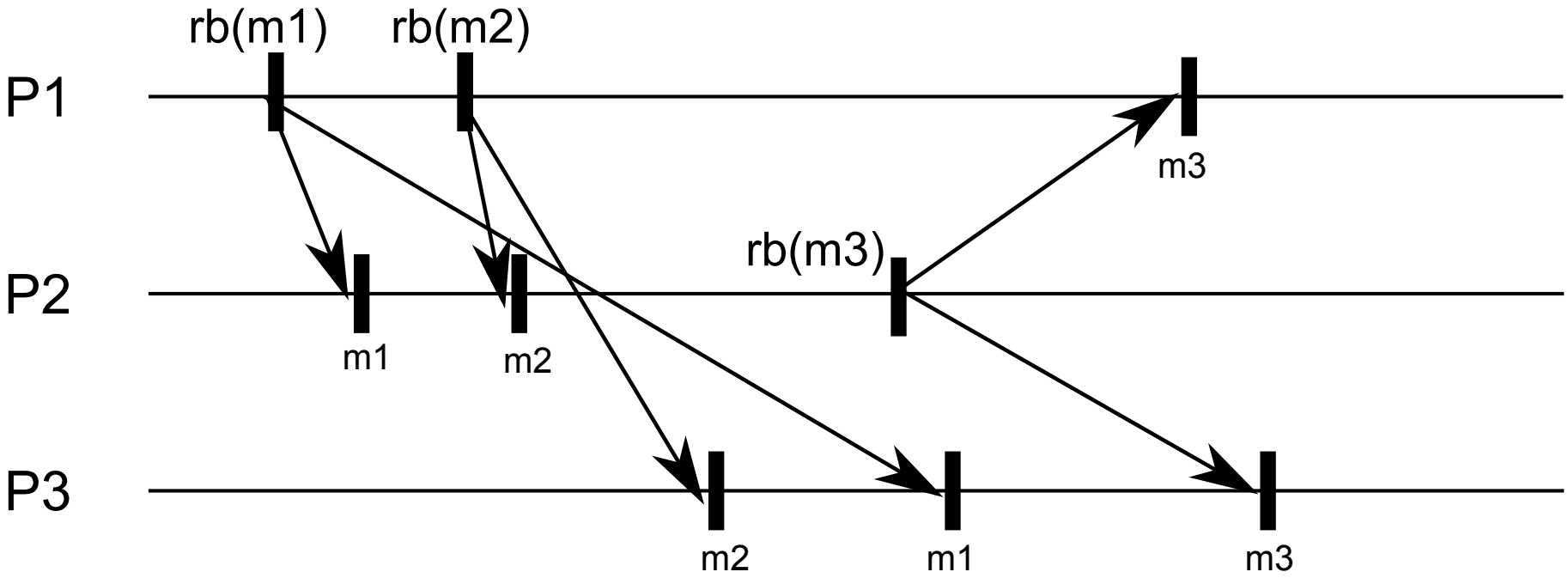
# FIFO broadcast (fbcast)

The same algorithm, less formal:

- Every message has a per-sender sequence number
- If the “right” message arrives: deliver it immediately
- If a message arrives out of order (at least one message with smaller sequence number is missing): Put message in queue and deliver it only after receiving and delivering the missing messages

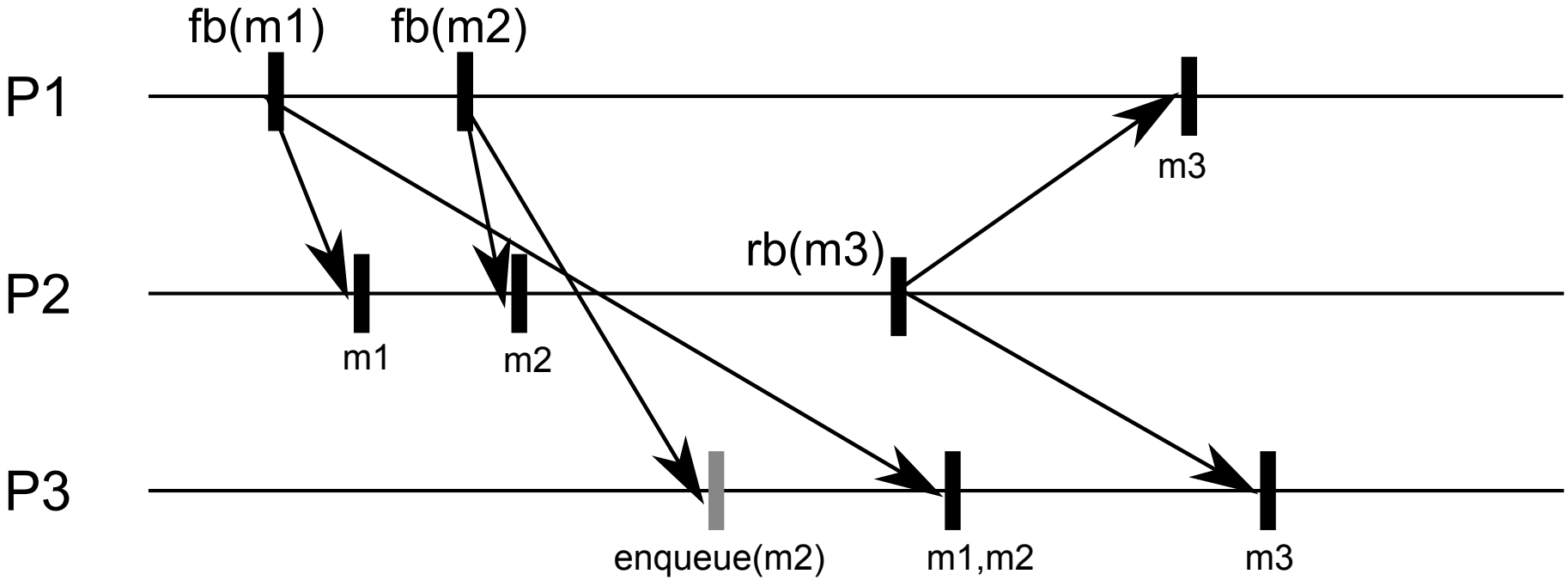
# rbcast vs. fbcast

- Reliable broadcast



# rbcast vs. fbcast

- FIFO broadcast




- P3 delivers *m*2 after *m*1

# Overview


1. Basics: Safety and Liveness

2. Broadcast
   - Best-effort broadcast
   - Reliable broadcast (rbcast)
   - FIFO broadcast (fbcast)
   - Causal broadcast (cbcast)
   - Atomic broadcast (abcast)

# Causal broadcast (cbcast)

Causal broadcast is a reliable broadcast that satisfies the following additional property:

## Properties of cbcast

- **Causal order:** If the broadcast of a message $m$ causally precedes the broadcast of a message $m'$ (i.e., if $m \rightarrow m'$), then no correct process delivers $m'$ unless it has previously delivered $m$.

# Causal broadcast (cbcast)

Causal broadcast is a reliable broadcast that satisfies the following additional property:

## Properties of cbcast

- **Causal order:** If the broadcast of a message $m$ causally precedes the broadcast of a message $m'$ (i.e., if $m \rightarrow m'$), then no correct process delivers $m'$ unless it has previously delivered $m$.

$\Rightarrow$ See lecture on logical clocks for a definition of causal precedence ($m_1 \rightarrow m_2$)

$\Rightarrow$ Note that causal order implies FIFO order

# Causal broadcast: simple algorithm

```
Uses: FIFO broadcast (fbcast)

upon event Init:
   rcntDeliver := ∅

upon event < CBroadcast, m >:
   trigger < fbcast.FBroadcast, <rcntDeliver || m> >;
   rcntDeliver := ∅


upon event < fbcast.FDeliver, <m_1,...,m_q> >:
   for i := 1...q:
       if p has not previously executed Cdeliver(m_i):
          trigger < Cdeliver, m_i >
          rcntDeliver := rcntDeliver || m_i
```

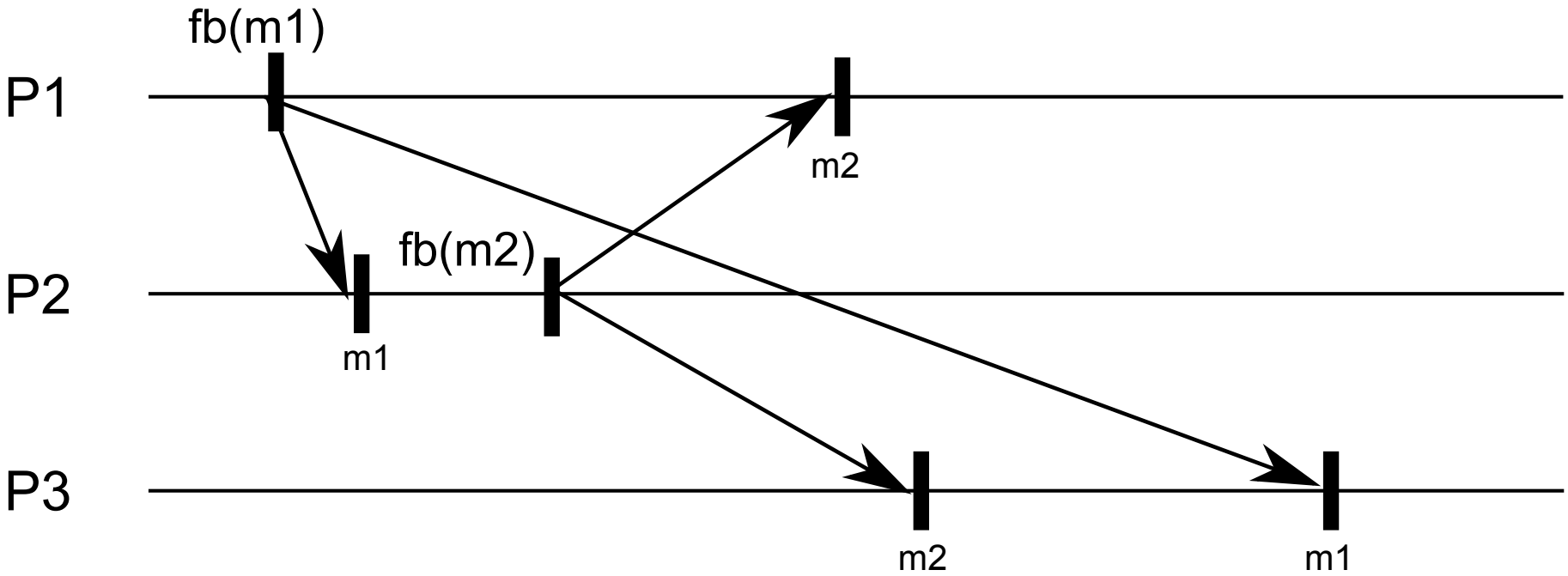
# Causal broadcast: simple algorithm

Basic idea:

- Uses FIFO broadcast $\Rightarrow$ guarantees correct FIFO order
- Records all local deliveries
- Instead of sending *m*, FIFO-broadcasts all locally delivered messages since last broadcast $\Rightarrow$ guarantees correct order

# fbcast vs. cbcast

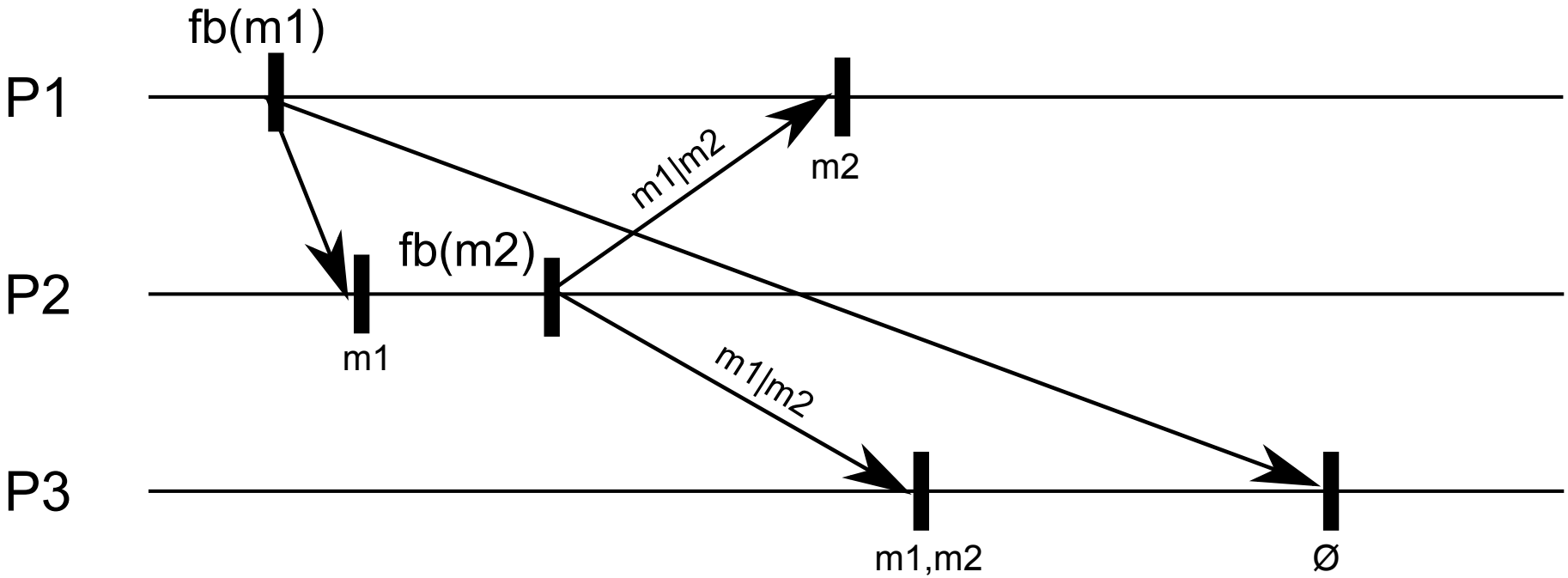
- FIFO broadcast




- P3 delivers *m*2 before *m*1, which violates causal order

# fbcast vs. cbcast

- causal broadcast




- P3 delivers *m*2 after *m*1

# Causal broadcast algorithms

- Previous algorithm involves a considerable overhead: each broadcast includes all causally preceding messages delivered since last broadcast
  - Benefit: algorithm is non-blocking
- Removing this overhead
  - Messages take only the IDs of those on which they depend (largest ID per process)
  - This is equivalent to labelling messages with a *vector clock*
  - Inconvenient: algorithm is blocking

# Overview


1 Basics: Safety and Liveness

2 Broadcast
- Best-effort broadcast
- Reliable broadcast (rbcast)
- FIFO broadcast (fbcast)
- Causal broadcast (cbcast)
- Atomic broadcast (abcast)

# Atomic broadcast (abcasts)

Atomic broadcast or total-order broadcast: defined in terms of

- Reliable broadcast properties +
- Total order property

## Additional property of abcast

*Total order:* If correct processes p and q both deliver messages m and m', then p delivers m before m' only if q delivers m before m'

# Atomic broadcast

- Algorithm?

# Atomic broadcast

- Algorithm?
- There's no (deterministic) algorithm for atomic broadcast in a system model:
  - Asynchronous communication
  - A single (a priori unknown) node my fail (crash fault)
- !? (see later)

# More broadcasts

- Atomic FIFO broadcast
  - Combination of atomic broadcast with FIFO order
  - Note: Plain atomic broadcast does *not* necessarily imply FIFO order!
- Atomic causal broadcast
  - Combination of atomic broadcast with causal order

# Summary of specifications

- *Validity:* If a correct process broadcasts a message $m$, then it eventually delivers $m$.
- *Agreement:* If a correct process delivers a message $m$, then all correct processes eventually deliver $m$.
- *Integrity:* For any message $m$, every correct process delivers $m$ at most once, and only if $m$ was previously broadcast by $sender(m)$.
- *FIFO Order:* If a process broadcasts a message $m$ before it broadcasts a message $m'$, then no correct process delivers $m'$ unless it has previously delivered $m$.
- *Causal Order:* If the broadcast of a message $m$ causally precedes the broadcast of a message $m'$, then no correct process delivers $m'$ unless it has previously delivered $m$.
- *Total Order:* If correct processes $p$ and $q$ both deliver messages $m$ and $m'$, then $p$ delivers $m$ before $m'$ if and only if $q$ delivers $m$ before $m'$.

# Summary of specifications

- Reliable broadcast = Validity + Agreement + Integrity
- FIFO broadcast = Reliable broadcast + FIFO order
- Causal broadcast = Reliable broadcast + Causal order

- Atomic broadcast = Reliable broadcast + Total order
- FIFO atomic broadcast = FIFO broadcast + Total order
- Causal atomic broadcast = Causal broadcast + Total order

# Summary of specifications

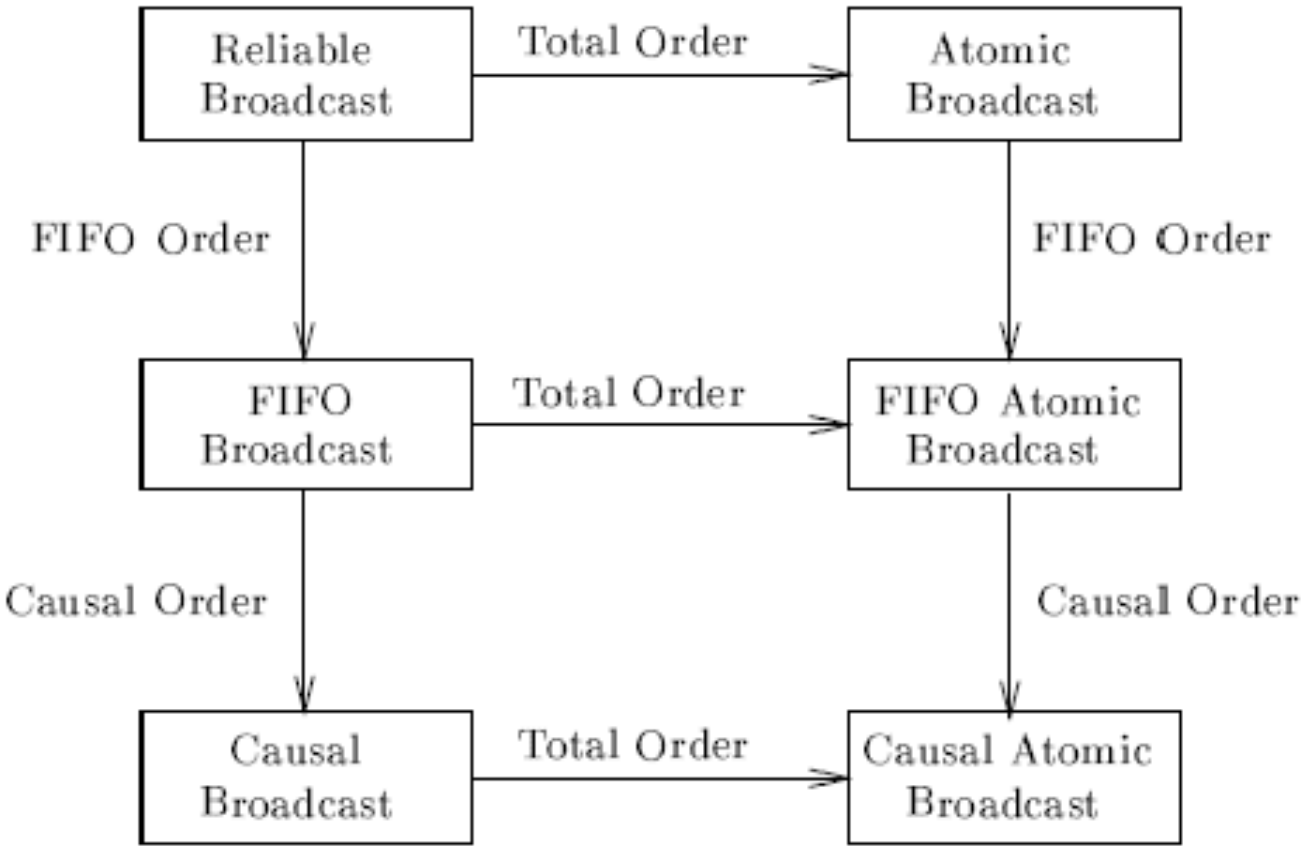



Figure 1: Relationship among Broadcast Primitives

# References I


Christian Cachin, Rachid Guerraoui, and Luís Rodrigues. *Introduction to Reliable and Secure Distributed Programming*. Springer, 2 edition, 2011.

Xavier Défago, André Schiper, and Péter Urbán. Total order broadcast and multicast algorithms: Taxonomy and survey. *ACM Computing Surveys*, 36(4):372–421, 2004.

Vassos Hadzilacos and Sam Toueg. A modular approach to fault-tolerant broadcasts and related problems. Technical Report TR94-1425, Department of Computer Science, Cornell University, 1994.